



Plano de Projeto: Avaliação, Benchmark e Fine-Tuning de Small Language Models (SLMs) para Automação Orçamentária Pública com n8n

Identificação do Projeto

- **Título do Projeto:** Assistente Inteligente para Orçamento Público Estadual baseado em SLMs Locais e Orquestração n8n
- **Instituição / Laboratório:** Universidade Estadual de Londrina (UEL) – Laboratório de Pesquisa em Gestão Pública (LABGEP / NIGEP)
- **Integrantes (Discentes):** Marcos Vinicius Beregula Ferreira, Guilherme Nascimento Braga
- **Colaboração Técnica:** Rafael Lançoni Santos (Desenvolvimento de Workflows n8n)
- **Coordenação e Orientação:** Prof. Dr. Daniel dos Santos Kaster (Departamento de Computação / UEL)
- **Horizonte de Execução:** Setembro de 2026 a Dezembro de 2026 (Prazo final: 31/12/2026)

1 Contexto, Motivação e Desafio Tecnológico

No âmbito da gestão governamental, os processos orçamentários do Estado do Paraná são regidos por um arcabouço normativo estrito, sustentado pela Lei Federal nº 4.320/1964 ([Brasil, 1964](#)), pela Lei de Responsabilidade Fiscal (LC nº 101/2000) e por regulamentações estaduais periódicas, destacando-se a Resolução SEFA nº 01/2026 ([Paraná. Secretaria de Estado da Fazenda, 2026b](#)) e o Manual Técnico do Orçamento (MTO/PR) ([Paraná. Secretaria de Estado da Fazenda, 2026a](#)). As rotinas diárias dos servidores públicos envolvem consultas exaustivas a classificadores orçamentários estruturados, tais como:

- **Natureza de Despesa:** Códigos hierárquicos de 6 a 8 dígitos (Categoria Econômica, Grupo, Modalidade de Aplicação, Elemento e Subelemento Detalhe);
- **Ações Orçamentárias:** Atividades, projetos e operações especiais (ex.: Ação 8000 – Encargos Especiais);
- **Fontes de Recursos:** Identificadores de destinação orçamentária vinculados aos sistemas SIAFIC e MTO;
- **Funções e Subfunções:** Agrupamentos setoriais da despesa pública (ex.: Subfunção 121 – Planejamento e Orçamento);
- **Dotação Adicional de Despesa (DAD):** Elaboração de justificativas formais técnico-administrativas de esgotamento de alternativas e preservação do equilíbrio orçamentário (Art. 18 da Resolução SEFA 01/2026).



Para automatizar o atendimento aos servidores, uma Prova de Conceito (PoC) inicial foi estruturada na plataforma de automação de fluxos **n8n**. Nos protótipos preliminares disponibilizados pela equipe técnica, o fluxo utiliza nós de agentes inteligentes integrados a ferramentas de consulta em banco relacional PostgreSQL e chamadas a modelos em nuvem (como Google Gemini e Groq/Llama-70B).

Contudo, a transição para um ambiente de produção institucional impõe desafios práticos determinantes:

1. **Mitigação de Custos Financeiros Recorrentes (Modelos Proprietários Pagos):**
A dependência de modelos de linguagem comerciais operados em nuvem acarreta custos contínuos e imprevisíveis tarifados por token (além da exigência de contratos em moeda estrangeira), o que compromete a sustentabilidade financeira do serviço público. A adoção de SLMs abertos locais elimina custos de faturamento por uso.
2. **Restrição Crítica de Hardware em Produção (Execução Exclusiva em CPU):**
O ecossistema de servidores do laboratório e dos órgãos parceiros não dispõe de GPUs dedicadas para processamento de inferência. O assistente deve rodar de forma perene e estável em processadores convencionais (CPU com instruções SIMD como AVX-512 ou ARM NEON).
3. **Inadmissibilidade Estrita de Alucinações ($H_{rate} = 0$):** Em rotinas orçamentárias públicas, códigos inexistentes ou classificações inventadas geram erros fatais em empenhos e liquidações fiscais. As respostas do modelo devem se ater com 100% de fidelidade aos registros retornados pelas consultas relacionais (Ji et al., 2023).
4. **Segurança, Governança e LGPD (Soberania de Dados):** Embora as leis e dotações orçamentárias sejam de caráter público, normas de governança pública e segurança da informação impedem que dados institucionais, históricos de solicitações e minutas de trabalho sejam trafegados ou armazenados em servidores no exterior sob controle de terceiros.

Para superar esses entraves, este projeto tem como objetivo central **pesquisar, avaliar mediante benchmarks padronizados e adaptar por Fine-Tuning supervisionado Small Language Models (SLMs) locais e quantizados**, integrando-os de ponta a ponta aos fluxos do n8n.

2 Fundamentação Teórica e Arquitetura do Sistema

2.1 Small Language Models (SLMs) e Leis de Escala

Historicamente, o paradigma de modelos fundacionais priorizou modelos massivos (de dezenas a centenas de bilhões de parâmetros) baseados na arquitetura Transformer decodificadora (Vaswani et al., 2017). No entanto, as leis de escala computacionalmente ótimas demonstradas no estudo *Chinchilla* (Hoffmann et al., 2022) evidenciaram que modelos de menor porte, quando expostos a conjuntos de dados substancialmente maiores e com alta densidade sintética (de 2 a 15 trilhões de tokens), atingem capacidade de raciocínio, aderência a instruções (*instruction following*) e uso de ferramentas equivalentes ou superiores a modelos massivos de gerações anteriores (Ben Allal et al., 2024; Yang et al., 2024).

2.2 Mecânica da Quantização e Inferência em CPU

Em modelos de linguagem, os parâmetros são originalmente representados em ponto flutuante de 16 bits (FP16 ou BF16), demandando aproximadamente 2 bytes por parâmetro. Para



viabilizar a execução em CPU comum, emprega-se a **Quantização Pós-Treinamento (PTQ)** baseada no ecossistema GGUF/llama.cpp (Gerganov and contributors, 2023):

- **Quantização por Blocos (k -quants):** No esquema Q4_K_M, os tensores de peso são particionados em superblocos com fatores de escala diferenciados, mantendo as camadas críticas de atenção (*attention layers*) em precisões intermediárias (5 e 6 bits) e as camadas lineares densas em 4 bits, minimizando a perda de perplexidade (Dettmers et al., 2022).
- **Gargalo de Largura de Banda de Memória (*Memory Bandwidth Bound*):** A fase de decodificação autoregressiva em CPU é limitada pela velocidade com que os pesos trafegam da memória RAM principal para os caches L1/L2/L3 do processador (Kim et al., 2023). Modelos quantizados reduzem o volume de tráfego por um fator de até 4×, aumentando substancialmente o throughput em tokens por segundo sem demandar aceleração de tensores em GPU.

2.3 Fundamentos Matemáticos da Adaptação de Baixo Posto (LoRA e QLoRA)

O Fine-Tuning completo de todos os pesos de uma rede (Θ) em CPU é proibitivo computacionalmente. Adota-se, portanto, a técnica de *Low-Rank Adaptation* (LoRA) (Hu et al., 2022), que congela a matriz de pesos pré-treinada $W_0 \in \mathbb{R}^{d \times k}$ e injeta matrizes de decomposição de baixo posto:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r}(B \cdot A) \quad (1)$$

onde $A \in \mathbb{R}^{r \times k}$ é inicializada por uma distribuição Gaussiana $\mathcal{N}(0, \sigma^2)$, $B \in \mathbb{R}^{d \times r}$ é inicializada em zero, $r \ll \min(d, k)$ representa o posto intrínseco de adaptação (tipicamente $r \in \{8, 16, 32\}$) e α é o hiperparâmetro de escala.

O método QLoRA (Dettmers et al., 2023) estende essa formulação aplicando o tipo de dados 4-bit *NormalFloat* (NF4), quantização dupla (*Double Quantization* – DQ) e otimizadores paginados (*Paged Optimizers*), reduzindo drasticamente o consumo de memória na fase de treinamento. Concluído o ajuste, os adaptadores ΔW são fundidos diretamente em W_0 (*weight merging*), permitindo a exportação para o formato GGUF sem introduzir qualquer sobrecarga computacional (*overhead*) de latência na inferência final.

2.4 Arquitetura Geral Integrada

A Figura 1 ilustra o fluxo operacional concebido. As consultas são recebidas por webhooks do n8n, processadas por nós de memória e roteadas aos subagentes. O SLM local realiza o parsing e a extração determinística de entidades e radicais, que alimentam as ferramentas SQL conectadas ao PostgreSQL. As respostas consolidadas são devolvidas ao servidor público em JSON parseável ou Markdown higienizado.

O diagrama completo da solução foi estruturado no formato nativo do Draw.io, localizado em docs/diagramas/fig.drawio, permitindo customizações visuais e exportação direta para a pasta de figuras (docs/fig/fig.pdf).

3 Metodologia Experimental e Roteiro de Execução

O projeto está delineado em quatro etapas metodológicas rigorosas, detalhadas a seguir de modo a fornecer uma referência técnica clara para implementação e replicação.

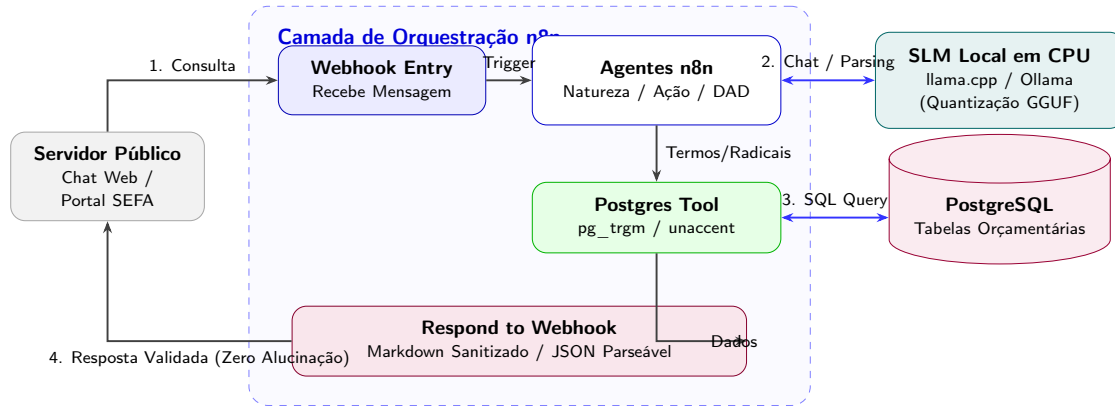


Figura 1: Visão Geral da Arquitetura Integrada: Usuário, Camada de Orquestração n8n, SLM Local em CPU e PostgreSQL.

3.1 Etapa 1: Pesquisa Sistemática e Triagem dos Modelos Candidatos

Seguindo as diretrizes acadêmicas formuladas pelo Prof. Daniel Kaster, serão selecionados modelos representativos da literatura atual em duas classes de operação:

1. **Classe A – Modelos Ultraleves (1B a 3B):** Projetados para execução instantânea em CPU com baixa ocupação de memória RAM (< 3,5 GB). Atuarão na triagem de mensagens, classificação de intenção (saudação vs. consulta direta) e extração determinística de entidades léxicas para chamadas de ferramentas (*tool calling*) (Schick et al., 2023). Os principais expoentes incluem o Qwen 2.5 1.5B/3B (Yang et al., 2024), Llama 3.2 1B/3B (Dubey et al., 2024) e SmolLM2 1.7B (Ben Allal et al., 2024).
2. **Classe B – Modelos Intermediários de Alta Capacidade (3.8B a 8B):** Focados em tarefas cognitivas complexas, como interpretação de dispositivos legais do MTO-PR, resolução de ambiguidades orçamentárias e síntese de minutas de DAD em JSON estrito sem desvios gramaticais (Willard and Louf, 2023). Destacam-se as famílias Phi-3.5/Phi-4-mini (Abdin et al., 2024), Gemma 2 (Gemma Team et al., 2024), Qwen 2.5 7B (Yang et al., 2024) e Llama 3.1 8B (Dubey et al., 2024).

3.2 Etapa 2: Framework de Benchmarking Empírico em CPU

A avaliação de desempenho será conduzida por meio de um script automatizado em Python executando testes locais via `llama.cpp server` e `Ollama`, alimentado por um conjunto canônico de 100 consultas reais extraídas das interações com o sistema orçamentário.

As métricas mensuradas dividem-se em duas categorias:

1. Métricas Computacionais de Sistema (CPU Hardware Metrics):

- **Throughput de Geração (T_{gen}):** Taxa de tokens por segundo durante a geração autoregressiva com $N \in \{4, 8\}$ threads de CPU:

$$T_{\text{gen}} = \frac{N_{\text{tokens}}}{t_{\text{geração}}} \quad [\text{tokens/s}] \quad (2)$$



Tabela 1: Matriz Comparativa dos SLMs Candidatos para Benchmark em CPU (quantização padrão GGUF Q4_K_M).

Modelo	Tamanho	Contexto	Quant.	RAM	Papel no Sistema / Atribuição
<i>Classe A: Modelos Ultraleves (< 3,5 GB RAM) – Triagem e Tool Calling</i>					
Qwen 2.5 1.5B	1,5B	32k	Q4_K_M	~ 1,4 GB	Extração determinística e parsing rápido
Llama 3.2 1B	1,2B	128k	Q4_K_M	~ 1,2 GB	Classificação de intenção e saudação
SmolLM2 1.7B	1,7B	8k	Q4_K_M	~ 1,5 GB	Extração de parâmetros e radicais
Qwen 2.5 3B	3,1B	32k	Q4_K_M	~ 2,6 GB	Tool calling estruturado e fallback
Llama 3.2 3B	3,2B	128k	Q4_K_M	~ 2,8 GB	Parsing semântico e roteamento
<i>Classe B: Modelos Intermediários – Raciocínio MTO e Minutas DAD</i>					
Phi-3.5-mini	3,8B	128k	Q4_K_M	~ 3,2 GB	Análise de conformidade MTO/PR
Phi-4-mini	3,8B	128k	Q4_K_M	~ 3,4 GB	Raciocínio lógico e justificativas fiscais
Gemma 2 2.6B	2,6B	8k	Q4_K_M	~ 2,3 GB	Modelo híbrido para síntese concisa
Qwen 2.5 7B	7,6B	128k	Q4_K_M	~ 5,8 GB	Síntese avançada de minutas de DAD
Llama 3.1 8B	8,0B	128k	Q4_K_M	~ 6,2 GB	Baseline teto comparativo (máxima precisão)

- **Time-To-First-Token (TTFT):** Intervalo de latência em segundos decorrido entre o envio do prompt com histórico de sessão e a emissão do primeiro token da resposta (avaliação do custo computacional do *prefill* na CPU).
- **Ocupação Máxima de Memória (*Peak RSS RAM*):** Consumo máximo de memória residente em GB aferido via chamadas de sistema (*getrusage*).

2. Métricas de Qualidade Normativa e Acurácia de Tarefa:

- **Acurácia de Triagem de Intenção (A_{int}):** Taxa de acerto binário na classificação de mensagens entre saudações (que devem emitir mensagem padrão imediata sem acionar o banco) e consultas operacionais de despesas.
- **Precisão na Extração de Entidades SQL (P_{sql}):** Conformidade estrita na extração dos três termos (**termo1**, **termo2**, **termo3**) e seus respectivos radicais morfológicos (ex.: conversão para substantivo singular, remoção de acentos diacríticos e isolamento de radicais com 4 a 5 caracteres).
- **Validade Sintática de Saída Estruturada (V_{json}):** Porcentagem de instâncias no fluxo de DAD que produzem objetos JSON válidos sem a presença de caracteres de formatação externos (*markdown fences* ou quebras espúrias).



- **Taxa de Alucinação Orçamentária (H_{rate}):** Frequência de respostas onde o modelo gera códigos numéricos inexistentes ou cria justificativas sem base factual nos dados retornados pela ferramenta (Ji et al., 2023):

$$H_{rate} = \frac{N_{respostas\ alucinadas}}{N_{total\ de\ consultas}} \times 100\% \quad (\text{meta: } 0\%) \quad (3)$$

A partir desses dados, será estabelecido o *ranking* que definirá os dois modelos finais (**Top-1 Leve** e **Top-1 Intermediário**).

3.3 Etapa 3: Curadoria de Dados e Fine-Tuning de Domínio (SFT via QLoRA)

A literatura demonstra que o ajuste supervisionado (*Supervised Fine-Tuning* – SFT) é indispensável para incutir o vocabulário e o estilo formal exigidos pela administração pública paranaense, reduzindo drasticamente o tamanho dos prompts do sistema e, por consequência, o tempo de processamento de prefill na CPU.

1. **Construção do Corpus de Ajuste Fino:** Estruturação de um conjunto com aproximadamente 1.500 pares de instrução-resposta no padrão ChatML ou Alpaca, contemplando:

- O Manual Técnico do Orçamento (MTO) e a classificação orçamentária da despesa pública do Paraná (Paraná. Secretaria de Estado da Fazenda, 2026a);
- O acervo de minutas e despachos da Resolução SEFA 01/2026 (Paraná. Secretaria de Estado da Fazenda, 2026b);
- Cenários explícitos de ausência de registros no PostgreSQL, condicionando o modelo a emitir respostas padronizadas de recusa ("*Não encontrei esta despesa no banco de dados*") em vez de inferir códigos inexistentes.

2. **Protocolo de Treinamento QLoRA:**

- **Hiperparâmetros de Adaptação:** Posto $r = 16$, fator de escala $\alpha = 32$ e taxa de descarte (*dropout*) de 0,05, aplicados a todos os módulos lineares de projeção (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj);
- **Otimização:** Otimizador AdamW de 8 bits (paged_adamw_8bit), taxa de aprendizado inicial $\eta = 2 \times 10^{-4}$ com decaimento por cosseno (*cosine schedule*) e 3 épocas completas de treinamento;
- **Fusão e Quantização Final:** Fusão dos pesos ΔW à matriz base W_0 com conversão subsequente via script `convert_hf_to_gguf.py` para quantização Q4_K_M, gerando os artefatos binários finais prontos para implantação em CPU.

3.4 Etapa 4: Integração Local no n8n e Validação Operacional

Na etapa final de implantação:

- O servidor de inferência local será inicializado como serviço contínuo via `llama.cpp server` expondo um endpoint HTTP compatível com a API da OpenAI na porta local 8080;
- Os nós de agentes no n8n serão reconfigurados para apontar para o nó OpenAI Chat Model com base URL <http://localhost:8080/v1>, com `temperature` fixada em 0,0 para tarefas determinísticas de extração SQL e 0,3 para redação de DAD;
- Serão executados testes de estresse de requisições concorrentes simulando o uso simultâneo por múltiplos técnicos orçamentários.



4 Cronograma e Plano de Ação

O plano de trabalho está organizado em cinco ciclos quinzenais até o encerramento do período letivo:

- **Ciclo 1 (08/09 a 22/09) – Preparação de Infraestrutura e Testbed:** Configuração do ambiente de execução local em CPU (1lama.cpp compilado com flags de vetorização AVX-512 e 01lama). Download e quantização dos candidatos da Tabela 1. Construção do script em Python para coleta de métricas e extração do dataset de 100 consultas orçamentárias. *Responsáveis: Marcos Beregula e Guilherme Nascimento.*
- **Ciclo 2 (23/09 a 07/10) – Execução dos Benchmarks e Seleção Top-2:** Execução integral da suite de testes de inferência em CPU. Tabulação comparativa de throughput (tokens/s), consumo de RAM, taxa de conformidade JSON e precisão na extração de radicais para PostgreSQL. Reunião de alinhamento com o orientador Prof. Daniel Kaster para ratificação dos modelos eleitos. *Responsáveis: Marcos Beregula, Guilherme Nascimento e Prof. Daniel Kaster.*
- **Ciclo 3 (08/10 a 05/11) – Engenharia de Dados e Fine-Tuning QLoRA:** Sistematização dos textos do MTO-PR e Resolução SEFA 01/2026 no formato de instrução supervisionada. Execução do treinamento dos adaptadores QLoRA para o modelo leve e o modelo intermediário. Validação de curvas de perda e fusão dos adaptadores para exportação em GGUF. *Responsáveis: Marcos Beregula e Guilherme Nascimento.*
- **Ciclo 4 (06/11 a 30/11) – Integração de Servidor e Homologação n8n:** Conexão dos modelos calibrados aos fluxos do n8n em substituição às APIs externas de nuvem. Realização de testes de validação com os fluxos de Natureza de Despesa, Ação Orçamentária e Minutas DAD. *Responsáveis: Toda a equipe (Marcos Beregula, Guilherme Nascimento, Rafael Lançoni e Prof. Daniel Kaster).*
- **Ciclo 5 (01/12 a 20/12) – Avaliação Final e Relatório de Extensão:** Consolidação dos resultados empíricos, redação do relatório final de extensão da UEL e entrega da documentação e repositório de código para os pesquisadores do LABGEP/NIGEP. *Responsáveis: Toda a equipe.*

Referências

- Abdin, M., Jacobs, S. A., Awan, A. A., Aneja, J., Awadallah, A., Awadalla, H., Bach, N., Bahree, A., Bakhtiari, A., Behl, H., et al. (2024). Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*.
- Ben Allal, L., Lozhkov, A., Bakouch, E., von Werra, L., and Wolf, T. (2024). Smollm2: When small language models go big on data. Hugging Face Technical Release: <https://huggingface.co/blog/smollm2>.
- Brasil (1964). Lei nº 4.320, de 17 de março de 1964. estatui normas gerais de direito financeiro para elaboração e controle dos orçamentos e balanços da união, dos estados, dos municípios e do distrito federal. Brasília: Presidência da República.
- Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. (2022). LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 30318–30332.



- Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 10088–10115.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. (2024). The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Gemma Team, Riviere, M., Pathak, S., Sessa, P. G., Smith, C., et al. (2024). Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*.
- Gerganov, G. and contributors (2023). llama.cpp: Port of facebook’s LLaMA model in C/C++ and GGUF specification. <https://github.com/ggerganov/llama.cpp>.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., et al. (2022). Training compute-optimal large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:30016–30030.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Dai, W., Madotto, A., and Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.
- Kim, S., Hooper, C., Wattanawong, T., Kang, M., Yan, R., Gholami, A., Mahoney, M. W., and Keutzer, K. (2023). Full stack optimization of transformer inference on CPUs. *IEEE Micro*, 43(5):71–80.
- Paraná. Secretaria de Estado da Fazenda (2026a). Manual técnico do orçamento (MTO) do estado do paraná. Curitiba: SEFA/PR. Disponível em: <https://www.fazenda.pr.gov.br/Pagina/Manual-Tecnico-do-Orcamento-MTO>.
- Paraná. Secretaria de Estado da Fazenda (2026b). Resolução SEFA nº 01/2026: Estabelece normas e procedimentos complementares para a execução orçamentária e financeira. Diário Oficial do Estado do Paraná.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 68539–68551.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30:5998–6008.
- Willard, B. T. and Louf, R. (2023). Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.
- Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Zhou, C., Li, C., Li, C., Liu, D., Huang, F., et al. (2024). Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.